

DEBACU AI

Integrador Universal de PMS

Arquitectura Técnica Completa

Versión

1.0 — Documento fundacional

Propósito

Arquitectura del integrador PMS→Debacu AI

Este documento define la arquitectura completa del integrador universal de PMS para Debacu AI: el sistema que permite a cualquier hotel conectar su software de gestión con la plataforma de inteligencia de riesgo y revenue de Debacu, de forma automática, segura y sin fricción.

1. VISIÓN ESTRATÉGICA

¿Qué es el integrador universal de PMS?

Debacu AI no es un PMS ni un RMS. Debacu AI es la capa de inteligencia que vive entre el PMS del hotel y las decisiones operativas del equipo de dirección.

Posición de mercado de Debacu AI

PMS (gestión operativa) → [DEBACU AI: riesgo + revenue intelligence] → Decisiones del hotel
Debacu lee del PMS. Nunca escribe en él. Nunca interfiere con la operación. Solo aporta inteligencia.

El integrador universal es el sistema que hace posible que cualquier hotel, independientemente del PMS que use, pueda conectarse a Debacu AI en minutos, sin intervención técnica por parte del equipo del hotel, y empezar a recibir:

- Scoring de riesgo de huéspedes actuales y futuras reservas
- Alertas del agente nocturno sobre perfiles de riesgo in-house
- Revenue intelligence: ADR, RevPAR, pickup, fuga de canal, anomalías
- Detección de patrones de fraude, no-shows y cancelaciones

El problema que resuelve

Hoy, Debacu funciona con importación manual de CSV. Eso funciona para algunos hoteles pioneros, pero no es escalable. La mayoría de hoteles no van a exportar un CSV cada día. Necesitan que su PMS hable con Debacu de forma automática, continua y sin que nadie tenga que hacer nada.

El integrador resuelve exactamente eso: el hotel elige su PMS en un desplegable, introduce sus credenciales una sola vez (o hace OAuth si el PMS lo soporta), y a partir de ese momento Debacu sincroniza automáticamente en background cada 15-30 minutos.

Lo que Debacu necesita del PMS — y solo eso

Debacu es read-only. No escribe en el PMS, no modifica reservas, no accede a configuración. Solo lee cuatro tipos de entidades:

Entidad PMS	Para qué la usa Debacu	Frecuencia de sync
Reservas	Riesgo canal, pickup, revenue, forecasting	Cada 15 min
Huéspedes	Scoring riesgo, detección fraude, perfiles	Cada 30 min
Estancias (in-house)	Agente nocturno, alertas en tiempo real	Cada 15 min
Tipos de habitación	Análisis de mix, RevPAR por categoría	Cada 4 horas

2. ARQUITECTURA DEL INTEGRADOR

Arquitectura: Hub-and-Spoke Universal

La arquitectura del integrador sigue el patrón hub-and-spoke: una capa canónica central que todos los PMS alimentan, y conectores específicos por PMS que traducen los datos al formato canónico de Debacu.

Principio fundamental

Añadir un PMS nuevo nunca cambia nada del resto del sistema. Solo se añade un conector nuevo. La capa canónica, las Edge Functions de sincronización, los modelos ML y los agentes IA permanecen exactamente igual.

Capas del sistema

Capa	Componente	Responsabilidad
1. PMS	apaleo / Tesipro / Cloudbeds / Mews / ...	Fuente de datos operativos del hotel
2. Conector	PMSConnector por proveedor (Edge Fn)	Auth, fetch, paginación, rate limiting
3. Raw layer	pms_raw_records (Supabase)	Datos tal como llegan, sin transformar
4. Mapper	PMSMapper por proveedor (TypeScript)	Traducción PMS → modelo canónico Debacu
5. Canónico	pms_reservations, pms_guests, pms_stays	Modelo universal normalizado
6. Feature layer	Pipeline ML (Python service)	Variables derivadas para los agentes IA
7. Agentes IA	Risk Agent, Revenue Agent, Nocturno...	Inteligencia sobre los datos canónicos

Flujo completo: de la API del PMS al agente nocturno

Este es el camino completo que recorre un dato desde que entra por la API del PMS hasta que el agente nocturno lanza una alerta:

1. El cron de Supabase dispara la Edge Function pms-sync-orchestrator cada 15 minutos
2. La función lee las conexiones activas de pms_connections (solo service_role)

3. Para cada conexión, llama a la Edge Function pms-sync-run con el proveedor y entidad correspondiente
4. pms-sync-run instancia el conector del PMS (ApaleoConnector, TesiproConnector, etc.)
5. El conector autentica contra el PMS (OAuth2 token refresh o API Key) y hace fetch paginado
6. Los datos raw se persisten en pms_raw_records con el payload original intacto
7. El mapper traduce cada registro raw al modelo canónico de Debacu
8. Los datos canónicos se insertan/actualizan en pms_reservations, pms_guests, pms_stays (UPSERT por external_id)
9. El Python ML Service lee las tablas canónicas y actualiza la feature layer
10. Los agentes IA leen la feature layer y producen scores, alertas y recomendaciones
11. El agente nocturno cruza estancias in-house con scoring de riesgo y lanza alertas al equipo del hotel

Reglas arquitectónicas no negociables

Stack

React + TypeScript + Vite (frontend) | Supabase Edge Functions (backend) | PostgreSQL Supabase (DB) | Python ML Service (IA) | Vercel (hosting)

- NUNCA se usan RPCs de Postgres. Toda lógica de negocio vive en Edge Functions
- La base de datos solo contiene tablas, índices, constraints y triggers de updated_at
- Toda consulta filtra siempre por org_id Y property_id — aislamiento estricto multi-tenant
- Todas las tablas tienen RLS habilitado
- Los campos PII de huéspedes (nombre, email, teléfono, documento) NUNCA se guardan en claro — se hashean como identity_key = HMAC(valor_normalizado + pepper)
- Las credenciales PMS se cifran en la Edge Function antes de persistir — nunca llegan al frontend en claro
- Debacu es read-only respecto al PMS — nunca escribe, nunca modifica

3. MODELO DE BASE DE DATOS

Tablas del integrador PMS

Tabla: pms_connections

Registro maestro de cada conexión PMS activa. Una organización puede tener múltiples conexiones (una por propiedad, o una multi-propiedad si el PMS lo permite).

```
CREATE TABLE pms_connections (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id            UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  property_id       UUID REFERENCES properties(id) ON DELETE SET NULL,
  provider_code     TEXT NOT NULL, -- 'APALEO' | 'TESIPRO' | 'CLOUDBEDS' | 'MEWS'
  environment       TEXT NOT NULL DEFAULT 'production', -- 'sandbox' | 'production'
  display_name      TEXT,          -- 'Hotel Costa Norte - apaleo'
  status            TEXT NOT NULL DEFAULT 'PENDING',
                  -- 'PENDING' | 'ACTIVE' | 'ERROR' | 'DISCONNECTED'
  last_sync_at      TIMESTAMPTZ,
  last_success_sync_at TIMESTAMPTZ,
  last_error_at     TIMESTAMPTZ,
  last_error_message TEXT,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Índices
CREATE INDEX idx_pms_connections_org ON pms_connections(org_id);
CREATE INDEX idx_pms_connections_property ON pms_connections(property_id);
CREATE INDEX idx_pms_connections_status ON pms_connections(status);

-- RLS
ALTER TABLE pms_connections ENABLE ROW LEVEL SECURITY;
-- Solo service_role accede directamente
-- El frontend consulta via Edge Functions con service_role
```

Tabla: pms_credentials

Almacena las credenciales cifradas. NUNCA accesible desde el frontend ni desde usuarios autenticados. Solo la Edge Function con service_role la lee, descifra en memoria, usa para el fetch al PMS, y descarta. La clave de cifrado vive en las variables de entorno de las Edge Functions, no en la base de datos.

```
CREATE TABLE pms_credentials (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  connection_id     UUID NOT NULL REFERENCES pms_connections(id) ON DELETE CASCADE,
  org_id            UUID NOT NULL, -- desnormalizado para RLS
```

```

-- Credenciales cifradas (AES-256-GCM en Edge Function antes de insertar)
encrypted_data TEXT NOT NULL, -- JSON cifrado: {client_id, client_secret,
token, refresh...}
iv
TEXT NOT NULL, -- vector de inicialización del cifrado
auth_mode
TEXT NOT NULL, -- 'OAUTH2' | 'API_KEY' | 'BASIC_AUTH'
expires_at
TIMESTAMPTZ, -- para OAuth2: cuándo expira el access token
created_at
TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at
TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- RLS: solo service role - ningún usuario autenticado puede leer esta tabla
ALTER TABLE pms_credentials ENABLE ROW LEVEL SECURITY;
CREATE POLICY pms_credentials_deny_all ON pms_credentials
FOR ALL TO authenticated USING (false);
-- Las Edge Functions usan service_role y bypasean RLS

```

Tabla: pms_sync_jobs

Log de cada ejecución de sincronización. Permite trazabilidad, debugging, retry automático y monitorización desde el panel admin.

```

CREATE TABLE pms_sync_jobs (
  id
  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  connection_id
  UUID NOT NULL REFERENCES pms_connections(id),
  org_id
  UUID NOT NULL,
  property_id
  UUID,
  entity_type
  TEXT NOT NULL, -- 'RESERVATION' | 'GUEST' | 'STAY' |
'ROOM_TYPE'
  sync_mode
  TEXT NOT NULL, -- 'FULL' | 'INCREMENTAL'
  status
  TEXT NOT NULL DEFAULT 'RUNNING',
  -- 'RUNNING' | 'SUCCESS' | 'PARTIAL' | 'FAILED'
  records_fetched
  INTEGER DEFAULT 0,
  records_inserted
  INTEGER DEFAULT 0,
  records_updated
  INTEGER DEFAULT 0,
  records_failed
  INTEGER DEFAULT 0,
  attempt_count
  INTEGER DEFAULT 1,
  next_cursor
  TEXT, -- para sincronización paginada incremental
  error_code
  TEXT,
  error_message
  TEXT,
  started_at
  TIMESTAMPTZ NOT NULL DEFAULT now(),
  finished_at
  TIMESTAMPTZ,
  duration_ms
  INTEGER
);

CREATE INDEX idx_pms_sync_jobs_connection ON pms_sync_jobs(connection_id,
started_at DESC);
CREATE INDEX idx_pms_sync_jobs_status ON pms_sync_jobs(status) WHERE status =
'RUNNING';

```

Tablas canónicas: pms_reservations y pms_guests

Estas son las tablas que alimentan directamente a los agentes IA. Contienen datos normalizados e independientes del PMS de origen.

```

CREATE TABLE pms_reservations (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  connection_id     UUID NOT NULL REFERENCES pms_connections(id),
  org_id            UUID NOT NULL,
  property_id       UUID NOT NULL,
  external_id       TEXT NOT NULL,    -- ID en el PMS
  -- Fechas
  check_in_date     DATE,
  check_out_date    DATE,
  booked_at         TIMESTAMPTZ,
  nights            INTEGER,
  -- Estado normalizado (independiente del PMS)
  status            TEXT NOT NULL,
  -- 'CONFIRMED' | 'CHECKED_IN' | 'CHECKED_OUT' | 'CANCELLED' | 'NO_SHOW' | 'PENDING'
  -- Canal y segmento
  channel_code      TEXT,    -- 'OTA_BOOKING' | 'DIRECT' | 'GDS' | ...
  channel_name      TEXT,
  segment_code      TEXT,
  rate_plan_code    TEXT,
  -- Revenue
  currency_code     TEXT,
  room_revenue_amount NUMERIC(12,2),
  total_amount      NUMERIC(12,2),
  -- Ocupantes
  adults            INTEGER,
  children           INTEGER,
  -- Relaciones
  external_guest_id TEXT,    -- referencia al guest en el PMS
  external_room_type_id TEXT,
  -- Metadatos
  source_updated_at TIMESTAMPTZ,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE(connection_id, external_id) -- idempotencia
);

CREATE TABLE pms_guests (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  connection_id     UUID NOT NULL REFERENCES pms_connections(id),
  org_id            UUID NOT NULL,
  property_id       UUID,
  external_id       TEXT NOT NULL,
  -- PII hasheado (NUNCA en claro) -- GDPR compliance
  identity_key      TEXT,    -- HMAC(nombre_normalizado + email + pepper)
  email_hash        TEXT,    -- HMAC(email_normalizado + pepper)
  phone_hash        TEXT,    -- HMAC(teléfono_normalizado + pepper)
  document_hash     TEXT,    -- HMAC(documento_normalizado + pepper)
  -- Datos no PII (sí en claro)
  country_code      TEXT,
  nationality_code   TEXT,
  is_company        BOOLEAN DEFAULT false,
  language_code     TEXT,
  -- Metadatos
  source_updated_at TIMESTAMPTZ,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE(connection_id, external_id)
);

```


4. EDGE FUNCTIONS

Mapa de Edge Functions del integrador

El integrador se implementa con las siguientes Edge Functions. Cada una tiene una responsabilidad única y bien definida.

Edge Function	Responsabilidad	Disparada por
pms-connection-save	Guarda y cifra credenciales del hotel	UI wizard (usuario)
pms-connection-test	Valida credenciales antes de activar conexión	UI wizard (usuario)
pms-sync-orchestrator	Lee conexiones activas y dispara sync por entidad	Cron cada 15 min
pms-sync-run	Ejecuta un ciclo de sync para una conexión+entidad	Orchestrator
pms-webhook-receiver	Recibe eventos push del PMS (si soporta webhooks)	PMS (push)
pms-connection-status	Devuelve estado de conexión al panel admin	Frontend
pms-sync-jobs-list	Lista historial de jobs para una conexión	Panel admin

pms-connection-save: guardado seguro de credenciales

Esta función es la única que escribe en `pms_credentials`. Recibe las credenciales desde el wizard, las cifra en memoria usando AES-256-GCM con una clave que vive solo en las variables de entorno de la Edge Function, y persiste el resultado cifrado. Las credenciales en claro nunca tocan la base de datos.

```
// Pseudocódigo - pms-connection-save
async function handler(req) {
  const { connectionId, orgId, credentials } = await req.json();
  // 1. Verificar que el usuario tiene permiso sobre esta org
  await assertUserBelongsToOrg(req, orgId);
  // 2. Cifrar credenciales en memoria
  const { encryptedData, iv } = await encrypt(
    JSON.stringify(credentials),
    Deno.env.get('PMS_ENCRYPTION_KEY')
  );
  // 3. Persistir solo el ciphertext - nunca el plaintext
  await supabaseAdmin
    .from('pms_credentials')
    .upsert({ connection_id: connectionId, org_id: orgId,
```

```
        encrypted_data: encryptedData, iv, auth_mode:
credentials.authMode });
    return { ok: true };
}
```

pms-sync-run: el corazón del integrador

Esta función implementa el patrón completo de sincronización: auth → fetch paginado → raw → mapper → canónico. Es agnóstica al PMS gracias al patrón strategy: recibe el provider_code y instancia el conector correspondiente.

```
// Pseudocódigo - pms-sync-run (flujo completo)
async function handler(req) {
    const { connectionId, entityType, syncMode } = await req.json();

    // 1. Leer conexión y credenciales (service_role, bypass RLS)
    const connection = await getConnection(connectionId);
    const credentials = await getAndDecryptCredentials(connectionId);

    // 2. Crear job de sync
    const jobId = await createSyncJob({ connectionId, entityType, syncMode });

    // 3. Instanciar el conector correcto según provider_code
    const connector = PmsProviderFactory.create(connection.provider_code,
credentials);

    // 4. Fetch paginado con retry y rate limiting
    let cursor = null; let total = 0;
    do {
        const page = await connector.fetch(entityType, { cursor, syncMode });
        // 5. Persistir raw
        await saveRawRecords(page.items, connection);
        // 6. Mapear a canónico e insertar (UPSERT por external_id)
        const canonical = page.items.map(item =>
            PmsMapperFactory.map(connection.provider_code, entityType, item, connection)
        );
        await upsertCanonical(entityType, canonical);
        cursor = page.nextCursor; total += page.items.length;
    } while (cursor);

    // 7. Cerrar job con éxito
    await finishSyncJob(jobId, { status: 'SUCCESS', records_fetched: total });
}
```

pms-webhook-receiver: events push (obligatorio desde día 0)

¿Por qué es obligatorio desde el principio?

Apaleo (y otros PMS modernos) piden declarar una URL de webhook durante el proceso de partner approval. Si esa URL no existe en el momento de la aprobación, el proceso se bloquea. En fase 1 el endpoint solo registra el evento — no lo procesa. Pero tiene que existir.

```
// pms-webhook-receiver - fase 1: solo registra, no procesa
async function handler(req) {
  const payload = await req.json();
  const provider = req.headers.get('X-PMS-Provider') ?? 'UNKNOWN';
  // Registrar el evento raw para futura referencia
  await supabaseAdmin.from('pms_webhook_events').insert({
    provider_code: provider,
    event_type: payload.eventType ?? 'UNKNOWN',
    payload: JSON.stringify(payload),
    received_at: new Date().toISOString(),
    processed: false // fase 2: se procesará aquí
  });
  return new Response('OK', { status: 200 }); // siempre 200 al PMS
}
```

5. CONECTORES PMS — ROADMAP

Qué PMS conectar y en qué orden

El orden de los conectores no es aleatorio. Está basado en tres criterios: impacto de mercado (cuántos hoteles objetivo usan ese PMS), calidad de la API (OAuth2 > API Key > SOAP), y accesibilidad del sandbox.

PMS	Fase	Auth	Sandbox	Mercado	Estado
apaleo	1 — Ahora	OAuth2	Público y gratuito	Boutique / nuevos	Construir
Tesipro/ Ulyses	2 — Con acceso	A confirmar	Solo con acuerdo	Cadenas ES	Con acceso
Cloudbeds	3	OAuth2	Accesible	Global medianos	Roadmap
Mews	3	OAuth2	Programa partner	Europa creciente	Roadmap
Beds24	4	API Key	Accesible	Independientes	Roadmap
Opera (Oracle)	5	OAuth2	Solo enterprise	Grandes cadenas	Futuro

Estructura del conector — patrón base

Todos los conectores implementan la misma interfaz. Eso garantiza que el orquestador funciona igual independientemente del PMS:

```
// supabase/functions/_shared/pms/connectors/PMSBaseConnector.ts
export interface PMSBaseConnector {
  // Autenticación — cada conector la implementa según su PMS
  authenticate(): Promise<void>;
  refreshTokenIfNeeded(): Promise<void>;

  // Fetch paginado de cada entidad
  fetchRoomTypes(opts: FetchOpts): Promise<PmsPagedResult<PmsRawEnvelope>>;
  fetchRooms(opts: FetchOpts): Promise<PmsPagedResult<PmsRawEnvelope>>;
  fetchGuests(opts: FetchOpts): Promise<PmsPagedResult<PmsRawEnvelope>>;
  fetchReservations(opts: FetchOpts): Promise<PmsPagedResult<PmsRawEnvelope>>;
  fetchStays(opts: FetchOpts): Promise<PmsPagedResult<PmsRawEnvelope>>;

  // Metadatos del conector
  providerCode: PmsProviderCode;
```

```

    rateLimit: { requestsPerMinute: number; requestsPerHour: number };
  }

  // Factory — añadir un PMS es añadir un case aquí
  export class PmsProviderFactory {
    static create(code: PmsProviderCode, credentials: any): PMSBaseConnector {
      switch (code) {
        case 'APALEO': return new ApaleoConnector(credentials);
        case 'TESIPRO': return new TesiproConnector(credentials);
        default: throw new Error(`PMS no soportado: ${code}`);
      }
    }
  }
}

```

ApaleoConnector — implementación OAuth2

```

// Flujo OAuth2 de apaleo
// 1. Client Credentials: POST https://identity.apaleo.com/connect/token
//   Body:
grant_type=client_credentials&client_id=X&client_secret=Y&scope=reservations.read
// 2. Access token con TTL de 1 hora → refresh automático
// 3. Todas las llamadas con: Authorization: Bearer {access_token}

// Endpoints que Debacu usa de apaleo:
// GET /v1/reservations?pageNumber=1&pageSize=100&dateFilter=Arrival
// GET /v1/guests?pageNumber=1&pageSize=100
// GET /v1/properties/{id}/unit-groups (room types)
// GET /v1/properties/{id}/units (rooms)
// GET /v1/folios?reservationId={id} (para revenue detallado)

// Rate limits apaleo: ~60 req/min por client — gestionar con backoff exponencial

```

6. WIZARD DE CONEXIÓN — UX DEL HOTEL

El hotel conecta su PMS en 4 pasos

El wizard es la interfaz que el hotel usa para conectar su PMS. El objetivo es que cualquier responsable de hotel (no necesariamente técnico) pueda completarlo en menos de 5 minutos. El hotel nunca debería necesitar ayuda del equipo de Debacu para conectarse.

Paso 1: Selección de PMS

- El hotel ve un desplegable con los PMS soportados por Debacu
- Cada PMS tiene logo, nombre comercial y una descripción de 1 línea
- Si el PMS del hotel no está en la lista, hay opción de 'solicitar integración'
- Al seleccionar el PMS, el wizard adapta el formulario al tipo de autenticación requerido

Paso 2: Credenciales (adaptativo según PMS)

Para OAuth2 (apaleo, Cloudbeds, Mews):

- El wizard muestra un botón 'Conectar con [PMS]' que abre el flujo OAuth del proveedor
- El hotel autoriza el acceso en la web del PMS y vuelve automáticamente a Debacu
- Debacu guarda el refresh token cifrado — el hotel no ve ni copia ninguna credencial

Para API Key (Beds24, otros):

- Formulario con campos: API Key, URL base (si aplica), identificador de propiedad
- Instrucciones inline: enlace directo a la sección del PMS donde generar la API Key

Paso 3: Validación automática

- Al continuar, Debacu llama a pms-connection-test en background
- La Edge Function intenta autenticar y hace una llamada de prueba mínima al PMS
- El hotel ve un indicador de progreso mientras se valida
- Si falla: mensaje de error claro con instrucciones de corrección (no stack traces)
- Si éxito: se activa la conexión y se programa el primer sync full

Paso 4: Sincronización inicial

- Debacu lanza el primer sync completo (full sync) en background
- El hotel ve el progreso: '2.847 reservas sincronizadas, 1.203 huéspedes...'
- Cuando termina, los módulos de riesgo y revenue se activan automáticamente
- El hotel recibe una notificación: 'Tu PMS está conectado. Debacu AI ya está trabajando.'

Principio UX del wizard

El hotel nunca debería necesitar abrir documentación técnica, preguntar al soporte de Debacu ni copiar strings complicados. Si hay algún paso que requiere eso, el wizard está mal diseñado.

7. DE LOS DATOS A LOS AGENTES IA

Cómo los datos del PMS alimentan a los agentes

Una vez que los datos canónicos están en Supabase, el Python ML Service los consume para generar la feature layer. Los agentes IA leen esa feature layer para producir inteligencia accionable.

Del PMS al agente de riesgo

Tabla canónica	Feature	Qué detecta el agente
pms_reservations	booking_window	Reservas de última hora = mayor riesgo no-show
pms_reservations	channel_code	OTA + precio bajo = perfil de riesgo mayor
pms_guests (cross-property)	repeat_incident_rate	Huésped con incidencias en otras propiedades del grupo
pms_stays (in-house)	stay_duration + status	Estancias anómalas, cambios de habitación frecuentes
pms_guests + incidencias Debacu	identity_key match	Huésped conocido con historial negativo

Del PMS al revenue intelligence

Tabla canónica	Feature	Qué genera el agente
pms_reservations	adr_by_channel	Comparativa de ADR por canal — detección de fuga
pms_reservations	channel_dependency_ratio	% de revenue que viene de una sola OTA = riesgo
pms_stays	occupancy_30d	Forecast de ocupación para los próximos 30 días
pms_reservations	booking_pace_7d	Pickup esta semana vs mismo período año anterior
pms_reservations	cancellation_ratio	Ratio de cancelación por canal y tipo de tarifa

El agente nocturno — caso de uso estrella

El agente nocturno es el ejemplo más potente de valor que el integrador PMS habilita. Sin la conexión automática, este agente no puede funcionar.

Qué hace el agente nocturno

A las 2:00 AM, el sistema consulta pms_stays donde stay_status = IN_HOUSE para esa propiedad. Cruza cada huésped in-house con su score de riesgo en Debacu, el historial de incidencias de la organización, y el perfil del canal de reserva. Si detecta un perfil de riesgo alto, genera una alerta que llega al recepcionista de turno vía push notification, email o integración con el sistema de mensajería del hotel.

Para que esto funcione, el agente nocturno necesita del integrador:

- Listado de estancias in-house actualizado (sync cada 15 min desde el PMS)
- identity_key del huésped para cruzar con el historial de incidencias de Debacu
- channel_code de la reserva (canal de riesgo más alto = alertas más tempranas)
- Número de noches restantes y fecha de checkout (urgencia de la alerta)

8. PLAN DE IMPLEMENTACIÓN

Orden de construcción — de cero al primer hotel conectado

Objetivo de este plan

Tener el primer hotel real conectado a apaleo, con datos sincronizándose automáticamente, y el agente nocturno produciendo alertas reales. Todo lo demás viene después.

Bloque 1: Base de datos y seguridad (1-2 días)

- 12. SQL completo de las 8 tablas del integrador con todas las columnas y tipos
- 13. Índices de performance en connection_id, org_id, property_id, status
- 14. RLS: pms_credentials con deny-all para authenticated, service_role bypass
- 15. RLS: pms_connections, pms_sync_jobs — visible solo para la propia org
- 16. RLS: pms_reservations, pms_guests, pms_stays — filtro org_id + property_id
- 17. Trigger updated_at en todas las tablas (solo esto — sin lógica de negocio)

Bloque 2: Edge Functions de credenciales (1 día)

- 18. pms-connection-save: cifrado AES-256-GCM + persistencia pms_credentials
- 19. pms-connection-test: validación de credenciales contra la API real del PMS
- 20. Variable de entorno PMS_ENCRYPTION_KEY en Supabase (no en código)

Bloque 3: Conector apaleo (2-3 días)

- 21. ApaleoConnector.ts: OAuth2 client credentials flow
- 22. Token refresh automático cuando TTL < 5 minutos
- 23. fetchReservations() con paginación y filtro por fecha modificación
- 24. fetchGuests() con paginación
- 25. fetchStays() para in-house actuales
- 26. Rate limiting: máximo 50 req/min con backoff exponencial en 429

Bloque 4: Mapper apaleo (1-2 días)

- 27. ApaleoReservationMapper: campos apaleo → CanonicalReservation
- 28. ApaleoGuestMapper: campos apaleo → CanonicalGuest + HMAC de PII
- 29. ApaleoStayMapper: campos apaleo → CanonicalStay
- 30. Catálogo de estados: mapeo exhaustivo reservation_status apaleo → status Debacku
- 31. Tests unitarios de cada mapper con payloads reales del sandbox

Bloque 5: Orchestrator de sync (1 día)

- 32. pms-sync-orchestrator: leer pms_connections activas y disparar sync por entidad
- 33. pms-sync-run: flujo completo raw → mapper → canónico con gestión de jobs
- 34. Cron en Supabase: cada 15 min para reservas/estancias, 30 min para huéspedes
- 35. pms-webhook-receiver: endpoint stub que registra eventos (no procesa)

Bloque 6: UI wizard (1-2 días)

- 36. Pantalla de selección de PMS con logos y descripciones
- 37. Formulario OAuth2 para apaleo (botón de autorización)
- 38. Llamada a pms-connection-test con indicador de progreso
- 39. Pantalla de sync inicial con contadores en tiempo real
- 40. Panel de estado de conexión (último sync, errores, jobs recientes)

Bloque 7: Verificación y primer hotel real (1 día)

- 41. Sync completo verificado: room_types, rooms, guests, reservations, stays
- 42. Verificar aislamiento: org A no puede ver datos de org B
- 43. Verificar GDPR: confirmar que ningún campo PII está en claro en la DB
- 44. Conectar primer hotel real con apaleo en sandbox
- 45. Activar agente nocturno con datos reales

Post-lanzamiento — Bloque 8

Con el primer hotel conectado y funcionando: (1) Conectar tablas canónicas al Python ML Service para feature engineering. (2) Activar revenue intelligence con datos reales. (3) Iniciar proceso de partner con Tesipro si hay acceso al sandbox. (4) Diseñar rate limiting distribuido para multi-hotel.

9. RIESGOS Y DECISIONES PENDIENTES

Riesgos y lo que queda por decidir

Riesgos críticos

Riesgo	Probabilidad × Impacto	Mitigación
PMS sin sandbox accesible (Tesipro)	Media × Alto	Esperar acceso real antes de construir el conector
Rate limit del PMS con muchos hoteles	Alta × Medio	Cola distribuida con espaciado temporal entre syncs
Webhook bloqueante para aprobación apaleo	Alta × Alto	Crear endpoint stub desde el primer día
PII en claro si el mapper falla antes del HMAC	Baja × Crítico	Hash en el mapper, nunca en la DB. Tests obligatorios.
Fuga de datos entre tenants por RLS incompleto	Baja × Crítico	Auditoría RLS antes de activar primer hotel real

Decisiones ya tomadas

- Primer PMS: apaleo (OAuth2, sandbox público, empezar hoy)
- Segundo PMS: Tesipro solo con acceso real al sandbox confirmado
- Credenciales: tabla pms_credentials con RLS deny-all + cifrado en Edge Function
- Onboarding: wizard UI, no configuración manual por el equipo Debacu
- Debacu es read-only respecto al PMS en todas las fases
- Agente nocturno como caso de uso estrella para demostrar valor

Pendiente de decidir

- Rate limiting distribuido: ¿cola en Supabase? ¿Redis externo? ¿Upstash?
- Canónico → feature layer: ¿trigger en Supabase o cron del Python Service?
- Notificaciones del agente nocturno: ¿push? ¿email? ¿Slack webhook del hotel?
- Panel de administración PMS: ¿accesible para el hotel o solo para Debacu team?

DEBACU AI — Documento técnico interno
Integrador Universal PMS v1.0